

Sisteme de Operare

Cap 5. Thread-uri și concurență

June 20, 2025

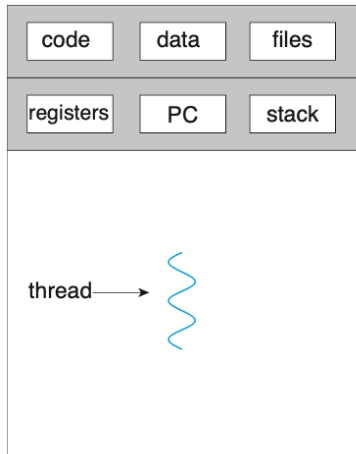
- 1 Thread-uri
- 2 Programarea multi-core
- 3 Modele de multithreading
- 4 Tipuri de threading
- 5 Probleme asociate cu threading-ul

- 1 Thread-uri
- 2 Programarea multi-core
- 3 Modele de multithreading
- 4 Tipuri de threading
- 5 Probleme asociate cu threading-ul

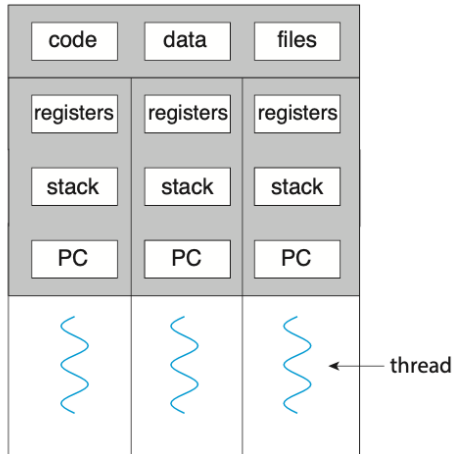
Noțiunea de thread

- un **thread**, denumit și **fir de execuție**, este unitatea elementară de utilizare a procesorului
- un proces poate să conțină unul sau mai multe thread-uri
- thread-ul este alcătuit din thread ID, program counter (PC) (sau instruction pointer), setul de regiștri cu valorile lor și stiva cu un anumit conținut
- un thread partajează cu celelalte thread-uri ale procesului secțiunea de cod (text), secțiunea de date și celelalte resurse ale SO, precum descriptorii de fișiere deschise sau semnalele
- un proces care are un singur flux de instrucțiuni este denumit proces **single-threaded**
- un proces cu mai multe fluxuri de instrucțiuni poate executa mai multe task-uri concomitent, este **multithreaded**

Procese single- și multi- threaded



single-threaded process



multithreaded process

Motivație

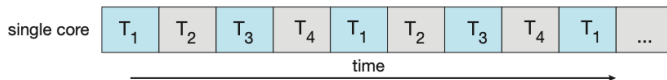
- aplicațiile software moderne sunt multithreaded; exemple:
 - o aplicație ce generează thumbnail-uri din fotografii
 - un web browser folosește thread-uri diferite pentru afișare respectiv aducere date din rețea
 - un procesor de text folosește un thread pentru afișare, un altul pentru captura de text și un al treilea pentru spell checker
 - un server de web folosește un thread pentru a asculta un port la care se conectează clienți noi, iar deservirea e realizată de un thread separat - fiecare client este deservit de un alt thread
- aplicațiile pot folosi puterea CPU-urilor multi-core prin proiectarea lor multithreaded; fiecare thread al aplicației are șansa să se execute pe un core separat, accelerând-o
- thread-urile sunt denumite și **light-weight processes**; thread-urile, spre deosebire de procese, au avantajul creării mai rapide - față de `fork()` - iar comunicația se face prin secțiunea de date comună, nu necesită mecanisme ale SO pentru transfer de date

Beneficii

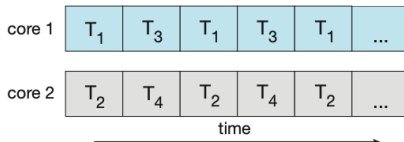
- **timp de răspuns mic:** execuția operațiilor lente se poate face pe fire separate, iar firul principal să se ocupe de interfața cu utilizatorul; spre deosebire, aplicația single-threaded se poate 'agăța'
- **partajarea resurselor:** aplicația multi-proces necesită suportul SO pentru transfer de mesaje sau memorie partajată; thread-urile au acces direct la aceleași zone de memorie, respectiv au secțiunea de cod comună - operează în același spațiu de adrese
- **economie:** alocarea resurselor unui proces e costisitoare; dacă avem mai multe thread-uri, schimbarea de context între thread-uri (și nu între procese) nu necesită salvarea/restaurarea întregului proces, ci doar registrul PC și stiva; comutarea între thread-uri e mai rapidă decât cea între procese
- **scalabilitate:** o aplicație multithreaded poate beneficia de avantajele execuției în paralel a activităților sale pe un CPU multi-core

- 1 Thread-uri
- 2 Programarea multi-core**
- 3 Modele de multithreading
- 4 Tipuri de threading
- 5 Probleme asociate cu threading-ul

Concurență și paralelism



- pe un CPU single-core, pentru o aplicație cu 4 thread-uri, **concurența** înseamnă că execuția lor va fi întrețesută în timp (sus)
- pe un CPU dual-core, execuția thread-urilor se face în **paralel** (jos)
- un sistem **concurrent** suportă execuția mai multor task-uri, permițând tuturor acestora să progreseze
- este posibil să avem concurență fără paralelism
- înaintea apariției sistemelor multi-core și multi-procesor, sistemele aveau doar o unitate de execuție, iar iluzia paralelismului era creată de comutarea rapidă între procese - rulare concurentă dar nu paralelă



Aspecte ale programării multi-core

- ➊ **identificarea task-urilor:** examinarea aplicației pentru a vedea ce task-uri se pot executa concurent/în paralel (ex. task-urile independente)
- ➋ **echilibrarea:** task-urile identificate trebuie să realizeze aproximativ același volum de muncă
- ➌ **partiționarea datelor:** uneori datele procesate pot fi partiționate, iar task-uri cu coduri identice vor procesa date diferite
- ➍ **dependințe de date:** dacă datele produse de un task servesc ca intrare pentru alt task, task-urile depind unele de altele prin date
- ➎ **testarea și depanarea:** un program multithreaded executat pe mai multe core-uri (sau chiar și pe același core, concurent) se poate executa în mai multe feluri; două rulări pot avea scenarii de execuție diferite chiar dacă au exact aceleași date de intrare; testarea și depanarea programelor multi-threaded este inerent mai dificilă decât cele single-threaded

Legea lui Amdahl (1)

- formalizează câștigul de performanță obținut prin adăugarea de core-uri suplimentare la execuția unei aplicații
- informal, exprimă "de câte ori" se execută mai repede aplicația pe N core-uri față de aplicația executată pe un singur core (serială)
- aplicația are o componentă inerent serială (secvențială) care nu se poate executa în paralel; se notează cu s această fracție serială
- timpul de execuție t al aplicației poate fi exprimat ca sumă între componenta serială și cea care poate fi paralelizată:

$$t_{\text{serial}} = s \cdot t + (1 - s) \cdot t \quad (1)$$

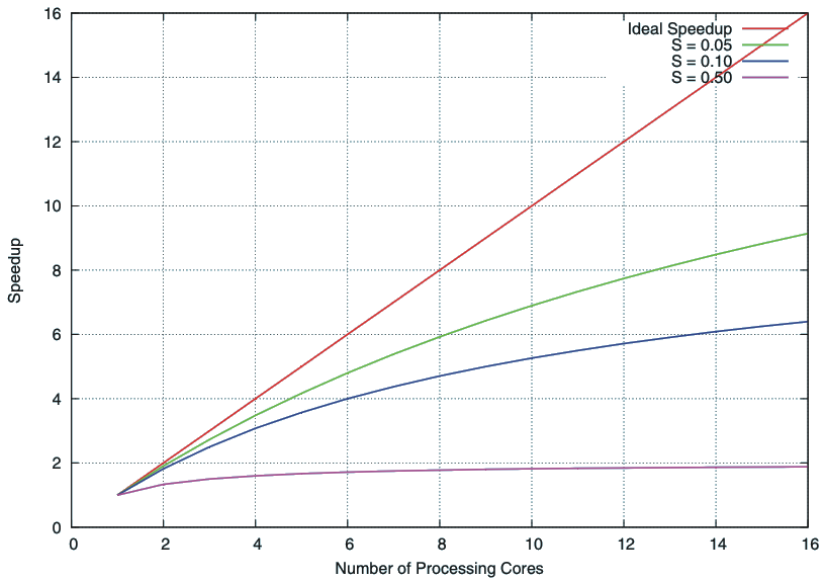
- folosind N core-uri, componenta paralelă se execută de N ori mai repede:

$$t_{\text{paralel}} = s \cdot t + (1 - s)/N \cdot t \quad (2)$$

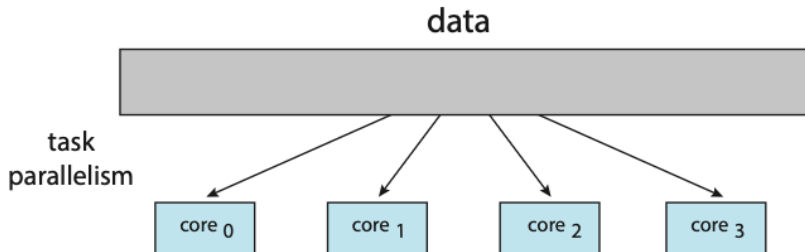
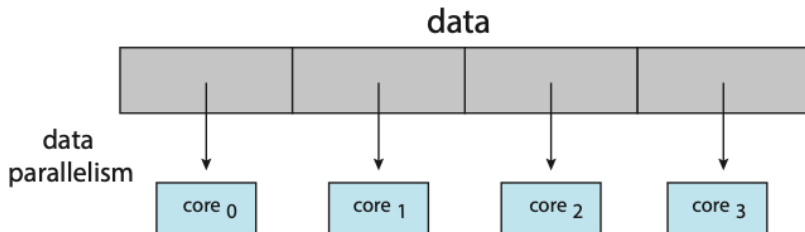
- legea lui Amdahl definește accelerarea programului paralelizat:

$$\frac{t_{\text{serial}}}{t_{\text{paralel}}} = \text{speedup} \leq \frac{1}{s + \frac{1-s}{N}} \quad (3)$$

Legea lui Amdahl (2)



Paralelism creat de date respectiv de task-uri



Exemplu de paralelism de date

- **data parallelism** înseamnă că datele sunt partiționate și pe fiecare partiție se aplică același task
- abordarea secvențială a adunării numerelor dintr-un array $a[1 \dots N]$
- **paralelizarea** prin date a adunării se poate face cu două core-uri:
 - primul core adună numerele $a[0], a[1] \dots a[N/2 - 1]$
 - al doilea face suma numerelor $a[N/2] \dots a[N]$
- adunarea nu merge chiar de două ori mai repede, trebuie să se facă suma rezultatelor parțiale
- **task parallelism** presupune ca fiecare task să fie diferit, iar ele să fie independente (datele lor să nu depindă unele de celelalte)

- 1 Thread-uri
- 2 Programarea multi-core
- 3 Modele de multithreading**
- 4 Tipuri de threading
- 5 Probleme asociate cu threading-ul

- **thread-urile utilizator** sunt modelele propuse spre execuție de către utilizator
- **thread-urile kernel** reprezintă suportul SO pentru execuția mai multor fire de către kernel
- există mai multe modele de threading:
 - **many to one**: mai multe thread-uri utilizator sunt mapate pe un singur thread kernel; există concurență, dar nu paralelism - thread-urile utilizator rulează pe un singur core; un thread care apelează un apel sistem blochează întregul proces
 - **one to one**: fiecare thread utilizator este executat de un thread kernel; proliferarea thread-urilor încetinește performanța sistemului; este implementarea curentă în Linux și Windows
 - **many to many**: un set de thread-uri user sunt mapate unui set de thread-uri kernel; numărul de thread-uri kernel alocate poate diferi de la o mașină la alta; se previne astfel proliferarea thread-urilor kernel

Biblioteci de thread-uri

- furnizează programatorului de aplicații un API pentru lucrul cu thread-uri
- implementarea acestora se poate face fie în user mode, fie în kernel mode, unde fiecare funcție API se transpune într-un system call
- există trei abordări folosite: POSIX Pthreads, Windows și Java
- Pthreads este o implementare a extensiei pentru thread-uri a POSIX, și există implementări atât în spațiul utilizator cât și în cel kernel
- Pthreads este o specificație, nu o implementare de bibliotecă de threading
- Windows implementează suportul pentru thread-uri la nivel kernel
- Java oferă direct suport programelor Java de a fi implementate multi-threading; JVM folosește în acest scop serviciile sistemului de operare pe care rulează

Pthreads API

```
#include <pthread.h>
#include <stdio.h>

#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    /* set the default attributes of the thread */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid, NULL);

    printf("sum = %d\n", sum);
}

/* The thread will execute in this function */
void *runner(void *param)
{
    int i, upper = atoi(param);
    sum = 0;

    for (i = 1; i <= upper; i++)
        sum += i;

    pthread_exit(0);
}
```

Windows threads API

```
#include <windows.h>
#include <stdio.h>
DWORD Sum; /* data is shared by the thread(s) */

/* The thread will execute in this function */
DWORD WINAPI Summation(LPVOID Param)
{
    DWORD Upper = *(DWORD*)Param;
    for (DWORD i = 1; i <= Upper; i++)
        Sum += i;
    return 0;
}

int main(int argc, char *argv[])
{
    DWORD ThreadId;
    HANDLE ThreadHandle;
    int Param;

    Param = atoi(argv[1]);
    /* create the thread */
    ThreadHandle = CreateThread(
        NULL, /* default security attributes */
        0, /* default stack size */
        Summation, /* thread function */
        &Param, /* parameter to thread function */
        0, /* default creation flags */
        &ThreadId); /* returns the thread identifier */

    /* now wait for the thread to finish */
    WaitForSingleObject(ThreadHandle, INFINITE);

    /* close the thread handle */
    CloseHandle(ThreadHandle);

    printf("sum = %d\n", Sum);
}
```

- 1 Thread-uri
- 2 Programarea multi-core
- 3 Modele de multithreading
- 4 Tipuri de threading**
- 5 Probleme asociate cu threading-ul

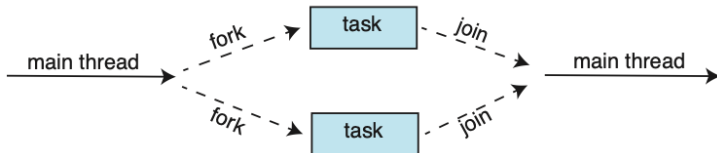
Threading implicit (1)

- sarcina de a crea, sincroniza și aștepta terminarea / opri thread-urile trece de la programator la compilator
- apelarea frecventă a funcțiilor de creare a thread-urilor este costisitoare; pe de altă parte nu este practic să creăm în avans un număr prea mare de thread-uri, ele cheltuie fără rost resursele sistemului
- soluția se numește **thread pool** (bazin de thread-uri), pre-alocarea unui număr constant de thread-uri ce așteaptă să li se dea de lucru: dacă sunt prea multe task-uri atunci toate thread-urile vor avea de lucru, la eliberarea unui thread el va prelua un task nou sau, dacă nu există task, el se va bloca (va intra în coada waiting), fiind trezit la apariția unui task; la terminare, thread-ul se va reîntoarce în pool

Threading implicit (2)

- avantajele unui thread pool:
 - ① satisfacerea unei cereri cu un thread existent se face mai repede decât prin crearea unui thread nou
 - ② un thread pool limitează numărul de thread-uri active la un moment dat
 - ③ separarea task-ului propriu-zis de mecanismului de creare a thread-ului permite diverse strategii pentru rularea task-ului; de exemplu, task-ul poate fi planificat să fie executat după un interval de timp sau să fie lansat în execuție periodic
- numărul de thread-uri din pool poate fi ajustat în funcție de numărul de core-uri de pe arhitectura unde se rulează, de memoria disponibilă, sau în funcție de tiparele de folosire
- modelul lui Apple de thread pool este denumit Grand Central Dispatch
- Windows API pune la dispoziție funcția `QueueUserWorkItem(&PoolFunction, NULL, 0)` pentru a executa un task reprezentat de funcția `DWORD WINAPI PoolFunction(PVOID Param) { ... }` cu ajutorul unui thread din pool

Strategia fork-join



- este o strategie **sincronă**, în care thread-ul principal așteaptă terminarea thread-urilor create
- se pretează pentru o implementarea threading-ului implicit, în care o bibliotecă este responsabilă pentru crearea thread pool-ului
- folosită de implementarea modelului de thread pool din Java

Modelarea fork-join în Java: divide et impera (1)

```
import java.util.concurrent.*;

public class SumTask extends RecursiveTask<Integer>
{
    static final int THRESHOLD = 1000;

    private int begin;
    private int end;
    private int[] array;

    public SumTask(int begin, int end, int[] array) {
        this.begin = begin;
        this.end = end;
        this.array = array;
    }

    protected Integer compute() {
        if (end - begin < THRESHOLD) {
            int sum = 0;
            for (int i = begin; i <= end; i++)
                sum += array[i];

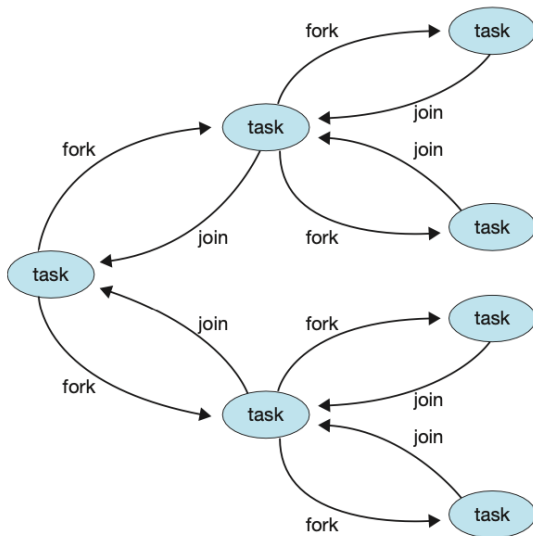
            return sum;
        }
        else {
            int mid = (begin + end) / 2;

            SumTask leftTask = new SumTask(begin, mid, array);
            SumTask rightTask = new SumTask(mid + 1, end, array);

            leftTask.fork();
            rightTask.fork();

            return rightTask.join() + leftTask.join();
        }
    }
}
```


Modelarea fork-join în Java (2)



OpenMP (1)

- este un **set de directive pentru compilator** și de asemenea un API pentru programe scrise în C, C++ sau FORTRAN ce furnizează suport pentru programare multi-core (paralelă) folosind shared memory
- OpenMP identifică **regiuni de cod paralelizabile** (care pot rula în paralel), la indicația utilizatorului

```
#include <omp.h>

int main(int argc, char *argv[]) {
    /* sequential code */
    #pragma omp parallel
    {
        printf("I am a parallel region.");
    }
    /* sequential code */
    return 0;
}
```

OpenMP (2)

- se creează atâtea thread-uri câte core-uri sunt în sistem
- blocul for este spart în thread-uri, iterațiile se pot executa și în paralel
- OpenMP este implementat de compilatoarele de C pentru Linux, Windows and macOS

```
#pragma omp parallel for  
for (i = 0; i < N; i++) {  
    c[i] = a[i] + b[i];  
}
```

Grand Central Dispatch (macOS)

- tehnologie dezvoltată de Apple pentru iOS și MacOS
- combinație dintre bibliotecă run-time, API și extensii de limbaj ce permite programatorilor să ruleze secțiuni de cod (task-uri) în paralel (similar cu OpenMP)
- GCD pune la dispoziție o coadă de task-uri denumită **dispatch queue**; task-urile sunt preluate de SO și executate cu ajutorul thread-urilor din thread pool; intern folosește POSIX Pthreads
- există două tipuri de dispatch queues: serial și concurent
- cozile seriale funcționează ca FIFO-uri: task-ul trebuie să se termine înainte ca următorul să fie luat din coadă; fiecare proces are propria coadă serială
- cozile concurente permit ca mai multe task-uri să poată fi luate din coadă, și executate paralel
- cozile concurente sunt globale sistemului și sunt grupate pe priorități: QOS_CLASS_USER_INTERACTIVE, QOS_CLASS_USER_INITIATED, QOS_CLASS_UTILITY, QOS_CLASS_BACKGROUND

Intel Thread Building Blocks (TBB)

- bibliotecă de șabloane care permite implementarea de aplicații paralele în C++
- nu necesită suport din partea compilatorului; TBB mapează task-urile desemnate de programator să ruleze în paralel
- planificatorul TBB echilibrează încărcarea și este cache-aware, dând prioritate task-urilor care au deja datele aduse în cache, accelerând execuția
- primitive pentru paralelizarea buclelor, operații atomice, realizarea accesului exclusiv, structuri de date: hash map, coadă, vector

```
for (int i = 0; i < n; i++) {
    app(v[i]); }
// constructia echivalenta TBB:
parallel_for (size_t(0), n, [=](size_t i) { app(v[i]); });
```

- 1 Thread-uri
- 2 Programarea multi-core
- 3 Modele de multithreading
- 4 Tipuri de threading
- 5 Probleme asociate cu threading-ul**

Apelurile `fork()` și `exec()`

- apelul `fork()` creează un nou proces; întoarcerea din proces se realizează atât în procesul părinte cât și în procesul copil; ce se returnează?
- pentru apelul `fork()`: se duplează toate thread-urile sau doar thread-ul care a chemat `fork`? unele sisteme UNIX implementează ambele variante
- apelul `exec()` va înlocui codul cu programul specificat ca parametru, deci se înlocuiesc toate thread-urile; nu avem funcționalități diferite pentru cazul multithreaded

Tratarea semnalelor (1)

- un **semnal** este folosit în sistemele UNIX pentru notificarea unui proces că a apărut un eveniment anume
- semnalele pot fi asincrone (ex. Ctrl-C apăsător de utilizator) sau sincrone (ex. acces neautorizat la memorie sau împărțire la 0); semnalele sincrone sunt trimise aceluiași proces care a realizat operația ce a determinat apariția semnalului
- semnalele urmează următorul tipar:
 - 1 semnalul este generat de SO ca urmare a apariției unui eveniment;
 - 2 semnalul este trimis către proces;
 - 3 odată primit, semnalul trebuie tratat de către proces.
- semnalul poate fi tratat:
 - 1 un signal handler (procedură de tratare a semnalului) implicit;
 - 2 un signal handler definit de utilizator.
- unele semnalele pot fi ignorate de proces, altele nu (ex. SEGFAULT)

Tratarea semnalelor (2)

- semnalul poate fi livrat fie thread-ului care a determinat apariția sa (semnal sincron), respectiv către toate thread-urile (ex. Ctr-C, semnal asincron, dar nu este obligatoriu să fie trimis la toate)
- apelul sistem UNIX pentru trimiterea unui semnal:

```
kill(pid_t pid, int signal)
```

- semnalul asincron este livrat către primul thread care nu blochează acel semnal; apelul sistem de trimitere a unui semnal către un thread:

```
pthread_kill(pthread_t tid, int signal)
```

- Windows nu furnizează suport explicit pentru semnale; emularea lor se poate face folosind Asynchronous Procedure Calls (APC), ce permite definirea unei funcții care se apelează când un thread primește o notificare despre apariția unui eveniment anume

Oprirea thread-urilor (1)

- **thread cancellation** se referă la oprirea thread-urilor înainte de terminarea lor
- oprirea poate fi:
 - ① **asynchronous cancellation**: un thread este terminat de un alt thread care îl oprește;
 - ② **deferred cancellation**: thread-ul verifică periodic dacă e cazul să se termine, iar dacă da, se termină din inițiativă proprie.
- la oprirea asincronă, poate apărea problema de ne-eliberare a tuturor resurselor alocate de un thread; ex. thread-ul tocmai actualizează informații partajate cu alte thread-uri; oprirea asincronă a unui thread s-ar putea să nu dea loc toate resursele
- oprirea de tip deferred permite o oprire curată, cu dealocarea controlată a resurselor
- apelul `pthread_cancel()` este o indicație de oprire, oprirea efectivă depinde de modul în care thread-ul răspunde la aceste solicitări

Oprirea thread-urilor (2)

```
pthread_t tid;
```

```
/* creaza thread-ul */
```

```
pthread create(&tid, 0, worker, NULL);
```

```
. . .
```

```
/* trimite comanda de oprire a thread-ului */
```

```
pthread_cancel(tid);
```

```
/* asteapta terminarea thread-ului */
```

```
pthread_join(tid, NULL);
```

- funcția `pthread_join` blochează, întoarce doar după terminarea thread-ului

Oprirea thread-urilor (3)

- thread-ul poate să dezactiveze apelurile de oprire (cancel)
- în starea cu cancel-urile dezactivate, acestea nu sunt pierdute, ci păstrate pentru timpul când thread-ul reactivează primirea cancel-urilor
- apelurile de oprire (cancel) nu pot fi primite de thread oricând, ci doar în așa-numitele **cancellation points**, în apelurile de sistem care blochează (ex. `read()`, v. c-da man `pthread`s)
- se poate defini un cancellation point și de către utilizator, prin apelul de funcție `pthread_testcancel()`; dacă există cereri de oprire salvate, acest apel nu întoarce și thread-ul se termină

```
while (1) {
    /* activitate utila */
    . . .
    /* verifica daca exista o cerere de oprire */
    pthread_testcancel();
}
```

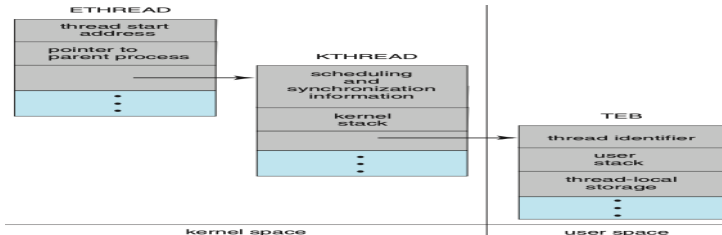
Thread Local Storage (TLS)

- thread-urile ce aparțin de un proces partajează aceleași date ale procesului
- TLS sunt datele declarate ca aparținând doar thread-ului în cauză; fiecare thread are propria copie a datelor
- sunt similare oarecum cu datele declarate ca fiind statice, doar că datele TLS sunt individuale thread-ului
- Java are construcția `ThreadLocal<T>`, Pthreads declară tipul `pthread_key_t`, ce individualizează variabila pentru fiecare thread, iar în Microsoft C# avem atributul de storage `[ThreadStatic]`
- compilatorul GCC furnizează cuvântul cheie `__thread` pentru declararea datelor TLS

```
static __thread int threadID;
```

Windows threads

- **contextul** thread-ului este dat de setul de regiștri, stivă și zona de stocare proprii thread-ului
- thread-urile Windows implică folosirea următoarelor structuri de date:
 - 1 ETHREAD (Executive THREAD block, stocat în kernel space) ce conține un pointer spre procesul de care aparține, adresa funcției de execuție a thread-ului, precum și un pointer la:
 - 2 KTHREAD (Kernel THREAD block), stocat tot în kernel space), conține informații de planificare și sincronizare, kernel stack (când thread-ul rulează în kernel mode are o altă stivă), precum și pointer la:
 - 3 TEB (Thread Environment Block, stocat în user space), conține thread ID, stiva thread-ului (IP, variabile locale etc.), și un array pentru TLS



Linux threads

- Linux furnizează și un apel de sistem denumit `clone()`; similar cu `fork()`, dar se trimite un set de flag-uri (indicatori) pentru a specifica ce structuri se partajează cu noul proces:
 - `CLONE_FS` - partajare informații despre directorul curent
 - `CLONE_VM` - partajare memorie
 - `CLONE_SIGHAND` - partajare handleri semnale
 - `CLONE_FILES` - partajare descriptori fișier
- dacă nu se specifică niciun flag, `clone()` e echivalentă cu `fork()`
- Linux definește structura kernel `struct task_struct` ce conține pointeri către structuri unde se stochează lista identificatorilor fișierelor deschise, signal handleri etc.; `fork` sau `clone` pot alege dacă să dublice aceste structuri sau să creeze doar legături